

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY BHUBANESWAR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FINAL PROJECT REPORT

TO-DO LIST COMMAND LINE APPLICATION

A DEMONSTRATION OF OBJECT-ORIENTED PROGRAMMING IN C++

NAME: SUJEET KUMAR BEHERA

ID: B25129

BRANCH: CSE-B

PROJECT: TO DO LIST (COMMAND LINE BASED)

ACADEMIC SESSION: 2025-2026

SUBMITTED TO

SHOBHANAYAK SRICHANDAN

1. INTRODUCTION & PROBLEM STATEMENT

1.1 Executive Summary

In software engineering, task management tools are standard exercises for learning control flow, data persistence, and software design. This project presents a full-fledged Command Line Interface (CLI) To-Do application built in modern C++. The project explicitly demonstrates core Object-Oriented Programming (OOP) paradigms through practical abstractions, including task scheduling, strict date validation, and automated recurring resetting mechanisms.

1.2 Problem Statement

Procedural programming models often struggle with state management, modular updates, and extensibility as system complexity scales. Traditional task management applications written procedurally rely on scattered arrays, global structures, and tight coupling between computational logic and file I/O operations.

Core Challenge: To design and implement a persistent, command-line-driven Task Management system in C++ that models real-world task behaviors (e.g., standard vs. recurring tasks, strict calendar date validation, and status transitions) while maintaining a clean object-oriented architecture.

1.3 System Scope & Constraints

- **Domain Requirements:** Dynamic handling of unique task IDs (`tid`), titles, detailed paragraph descriptions, validated due dates (`YYYY-MM-DD`), and completion states.
- **Polymorphic Requirement:** Differentiating standard tasks from recurring tasks. Recurring tasks must auto-reset to 'Pending' once their specified interval (2 to 30 days) has elapsed relative to the system date.
- **Interactive User Operations:** Supporting menu options for creating, sorting, searching, updating, status-reversing, and selectively deleting tasks.
- **Data Integrity & Persistence:** Disk storage via custom text file serialization (`tasks.txt`) ensuring complete state recovery across application runs.

1.4 Key Features Overview

Feature Module	Functional Description	OOP Mechanism
Task Hierarchy	Separates standard static tasks from dynamic auto-recurring task types.	Inheritance & Polymorphism
Date Validation	Verifies syntax, leap years, and ensures due dates strictly exceed today.	Encapsulated Utility Struct
Auto-Recurrence	Compares system calendar time against task intervals on launch to reset states.	Dynamic Method Overriding
File I/O Engine	Serializes object graphs into text records and parses them back on start.	Abstraction & Stream I/O

2. PROJECT OBJECTIVES & METHODOLOGY

2.1 Primary Objectives

- **Demonstrate Core OOP Pillars:** Showcase practical applications of Encapsulation, Abstraction, Single Inheritance, and Dynamic Polymorphism within a unified C++ codebase.
- **Enforce Strict Business Logic:** Implement real-time system date comparison to restrict task due dates to future dates and automate recurring task workflows.
- **Provide Flexible Filtering & Search:** Enable multi-criteria task sorting (by ID, by due date) and token-based string keyword searching across titles and detailed descriptions.
- **Ensure Safe Memory & State Persistence:** Eliminate raw pointer memory leaks using modern C++ smart pointers (`std::unique_ptr`) and implement disk serialization.

2.2 System Development Methodology

The development followed a modular, bottom-up design strategy structured into four distinct phases:

Phase I: Core Data Abstraction & Helper Engineering

Constructed the `Date` encapsulation unit to handle calendar arithmetic, leap-year checks, string parsing (`YYYY-MM-DD`), and system time extraction using C standard library headers (`<ctime>`).

Phase II: Domain Object Modeling (Class Hierarchy)

Designed the abstract base `Task` class containing protected member variables and virtual interface functions (`display()`, `serialize()`, `checkRecurringReset()`). Derived the `RecurringTask` sub-class to override state behavior.

Phase III: Storage & Controller Layer (`TodoManager`)

Engineered the centralized controller class holding a polymorphic container (`std::vector<std::unique_ptr<Task>>`). Implemented file reading/writing routines, ID auto-incrementing logic, and batch manipulation handlers.

Phase IV: Interactive CLI Interface & Input Validation

Developed a robust main loop driven by user menu selections, featuring strict input cleaning, buffer clearing, and error trapping for invalid types or missing IDs.

3. OOP CONCEPTS & SYLLABUS MAPPING

The architecture directly operationalizes core theoretical concepts from the academic OOP syllabus across all three prescribed modules.

3.1 Module-Wise Concept Traceability

Module I Concepts Implemented

- **Encapsulation & Access Control:** Member variables are marked `protected` or `private` to shield internal state from uncontrolled external modification. Access is strictly routed through public getters and setters.
- **Constructors & Destructors:** Explicit parameterized constructors handle member initialization lists, and virtual destructors guarantee clean memory release during object deletion.
- **Const Correctness:** Read-only member methods (`isCompleted() const`, `display() const`) are explicitly qualified with `const` to enforce compile-time state immutability.
- **Structures & Utilities:** Value objects like `Date` use C++ structures to package logically related attributes and member validation functions.

Module II Concepts Implemented

- **Single Class Inheritance:** `RecurringTask` inherits publicly from `Task`, inheriting base properties (`tid`, `title`, `description`) while extending functionality with `recurDays`.
- **Base Initialization in Derived Constructors:** The `RecurringTask` constructor delegates core attribute setup to its parent class constructor via member initializer syntax (`: Task(...)`).
- **Resource Management via Modern Pointers:** Replaced standard raw allocation with `std::unique_ptr` and `std::make_unique` to guarantee safe dynamic memory transfer and automatic deletion upon vector re-allocation.

Module III Concepts Implemented

- **Dynamic Polymorphism & Late Binding:** Base class pointer containers trigger sub-class specific methods dynamically at runtime via `virtual` method dispatch tables (vtables).
- **Pure Virtual Interface Patterns:** Virtual functions enable homogeneous handling of heterogeneous task object lists.
- **STL Containers, Iterators & Algorithms:** Extensive utilization of `std::vector` for dynamic storage, `std::set` for key indexing, and algorithm functions (`std::sort`, `std::find_if`, `std::remove_if`).
- **File Streams (Text I/O):** Text serialization using `std::ifstream` and `std::ofstream` with stringstream token parsing.

4. CODE ARCHITECTURE - MODULE ANALYSIS

4.1 Concept: Encapsulation & Data Validation (`Date` Struct)

The `Date` structure encapsulates calendar attributes (`year`, `month`, `day`) alongside internal logic to calculate leap years, validate date bounds, parse string inputs, and perform temporal comparisons against system time.

```
// Encapsulated Date structure with self-validation logic
struct Date {
    int year, month, day;

    bool isValid() const {
        if (month < 1 || month > 12 || day < 1 || day > 31) return false;
        if (month == 2) {
            bool isLeap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
            return day <= (isLeap ? 29 : 28);
        }
        if (month == 4 || month == 6 || month == 9 || month == 11) return day <= 30;
        return true;
    }

    bool isAfter(const Date& other) const {
        if (year != other.year) return year > other.year;
        if (month != other.month) return month > other.month;
        return day > other.day;
    }

    std::string toString() const {
        std::ostringstream ss;
        ss << year << "-" << std::setw(2) << std::setfill('0') << month
           << "-" << std::setw(2) << std::setfill('0') << day;
        return ss.str();
    }

    static Date parse(const std::string& str) {
        int y = 0, m = 0, d = 0; char d1, d2;
        std::istringstream ss(str);
        if (ss >> y >> d1 >> m >> d2 >> d && d1 == '-' && d2 == '-') return {y, m, d};
        return {0, 0, 0};
    }
};
```

Key Technical Takeaways:

- **Const Member Methods:** Methods like `isValid()` and `toString()` are marked `const` to guarantee zero side-effects during execution.
- **Static Parsing Utility:** `Date::parse()` encapsulates string extraction cleanly, returning invalid sentinel date instances on input errors.

4. CODE ARCHITECTURE - MODULE ANALYSIS (CONT.)

4.2 Concept: Class Hierarchy, Inheritance & Polymorphism

The core domain hierarchy features a base `Task` class and a derived `RecurringTask` class. Virtual methods ensure late binding during program runtime.

```
// Base Class representing standard tasks
class Task {
protected:
    int tid;
    std::string title;
    std::string description;
    Date dueDate;
    bool completed;

public:
    Task(int tid, std::string title, std::string description, Date dueDate, bool completed =
false)
        : tid(tid), title(title), description(description), dueDate(dueDate),
completed(completed) {}

    virtual ~Task() = default; // Virtual destructor for safe polymorphic cleanup

    virtual void checkRecurringReset() {} // Base hook for overriding
    virtual std::string getTypeString() const { return "Normal"; }

    virtual void display() const {
        std::cout << "Task ID: " << tid << " | Type: " << getTypeString()
                  << "\nTitle  : " << title << "\nDue    : " << dueDate.toString()
                  << "\nStatus : " << (completed ? "Completed" : "Pending") << "\n";
    }
};

// Derived Class implementing specific recurrence logic
class RecurringTask : public Task {
private:
    int recurDays;

public:
    RecurringTask(int tid, std::string title, std::string desc, Date dueDate, int recurDays,
bool completed = false)
        : Task(tid, title, desc, dueDate, completed), recurDays(recurDays) {}

    std::string getTypeString() const override { return "Recurring"; }

    void checkRecurringReset() override {
        if (completed) {
            Date today = Date::today();
            int daysSinceDue = today.daysDifference(dueDate);
            if (daysSinceDue >= 0 && (daysSinceDue % recurDays == 0 || daysSinceDue >=
recurDays)) {
                completed = false; // Reset status back to pending
                while (!dueDate.isAfter(today)) {
                    dueDate = dueDate.addDays(recurDays); // Advance due date
                }
            }
        }
    }
};
```

4. CODE ARCHITECTURE - MODULE ANALYSIS (CONT.)

4.3 Concept: Controller Logic & STL Container Management

The `TodoManager` class acts as an encapsulated controller holding a vector of polymorphic `std::unique_ptr<Task>` instances. STL algorithms perform sorting, search filtering, and record transformations.

```
// Manager Class handling dynamic task lifecycle and persistence
class TodoManager {
private:
    std::vector<std::unique_ptr<Task>> tasks;
    const std::string filename = "tasks.txt";

public:
    void checkAllRecurringTasks() {
        for (auto& task : tasks) {
            task->checkRecurringReset(); // Polymorphic method call
        }
    }

    void viewTasksSortedByDate() {
        std::vector<Task*> sortedList;
        for (const auto& t : tasks) sortedList.push_back(t.get());

        // Lambda-based STL Sorting
        std::sort(sortedList.begin(), sortedList.end(), [](Task* a, Task* b) {
            Date da = a->getDueDate(), db = b->getDueDate();
            if (da.year != db.year) return da.year < db.year;
            if (da.month != db.month) return da.month < db.month;
            return da.day < db.day;
        });

        for (auto task : sortedList) task->display();
    }

    void deleteTasksByIds(const std::set<int>& idsToDelete) {
        // Erase-remove idiom combined with STL algorithm
        tasks.erase(std::remove_if(tasks.begin(), tasks.end(), [&idsToDelete](const auto& t)
{
            return idsToDelete.count(t->getTid()) > 0;
        }), tasks.end());
    }
};
```

Architectural Benefits:

- **Smart Pointer Ownership:** `std::unique_ptr` ensures exclusive ownership semantics and avoids dangling pointer exceptions.
- **Predicate Lambdas:** Inline anonymous functions customize STL algorithm behaviors without polluting global namespace scope.

5. APPLICATION EXECUTION & OUTPUT DEMONSTRATION

This section presents terminal screen captures demonstrating core system features during execution.

5.1 Add Task Workflow & Date Validation

```
--- TO-DO COMMAND LINE APPLICATION --- 1. Add new task | 2. View task | 3. Delete task | 4. Search task | 5. Change status | 6. Update task | 7. Exit Select option: 1 --- Add New Task --- 1. Normal Task 2. Recurring Task Select task type (1-2): 2 Enter Title: Submit OOP Lab Report Enter Description: Complete the detailed documentation report for C++ CLI project. Enter Due Date (YYYY-MM-DD): 2024-01-10 Error: Due date must exceed today's date (2026-11-01). Try again. Enter Due Date (YYYY-MM-DD): 2026-11-15 Enter Recurring Duration in days (between 2 and 30): 7 Recurring Task added successfully with ID: 101
```

5.2 View Tasks (Sorted by Due Date)

```
Select option: 2 --- View Tasks --- a. All (sort by ID) b. All (sort by due date) c. Pending tasks only d. Completed tasks only Option: b ---------------------------------------- Task ID : 101 Type : Recurring Title : Submit OOP Lab Report Description : Complete the detailed documentation report for C++ CLI project. Due Date : 2026-11-15 Status : Pending Recur Days : Every 7 days ---------------------------------------- Task ID : 102 Type : Normal Title : Semester Exam Registration Description : Pay fees and verify course registration forms. Due Date : 2026-11-20 Status : Pending ----------------------------------------
```

5. APPLICATION EXECUTION & OUTPUT (CONT.)

5.3 Keyword Search & Multi-ID Status Modification

```
Select option: 4 --- Search Task --- Enter a single search keyword: Report [Match Found] Task ID: 101 | Details: Submit OOP Lab Report (2026-11-15) Select option: 5 --- Change Task Status --- Enter Task IDs separated by space to reverse status: 101 102 Task ID 101 status changed to: Completed Task ID 102 status changed to: Completed
```

5.4 Selective Task Update & File Persistence

```
Select option: 6 --- Update Task --- Enter Task ID to update: 101 Current Task Details: Task ID : 101 Title : Submit OOP Lab Report Description : Complete the detailed documentation report for C++ CLI project. Due Date : 2026-11-15 Enter new Title (or press Enter to keep old): Submit Final OOP Report Enter new Description (or press Enter to keep old): Enter new Due Date (YYYY-MM-DD) (or press Enter to keep old): 2026-11-18 Task updated successfully! Select option: 7 All tasks saved successfully to 'tasks.txt'. Exiting application.
```

5.5 Serialized File Format (`tasks.txt`)

```
RECURRING|101|Submit Final OOP Report|Complete the detailed documentation report for C++ CLI project.|2026-11-18|1|7
NORMAL|102|Semester Exam Registration|Pay fees and verify course registration forms.|2026-11-20|1
```

6. STRUCTURAL ANALYSIS & SYSTEM EVALUATION

6.1 Solid Design Principles Alignment

The software architecture aligns closely with established object-oriented design principles:

- **Single Responsibility Principle (SRP):** The `Date` struct handles calendar validation exclusively, `Task` handles attribute representations, `TodoManager` manages collection persistence, and `main()` handles the user input loop.
- **Open/Closed Principle (OCP):** New specialized task types (e.g., `UrgentTask` with priority scores) can be added by deriving from `Task` without modifying existing controller methods.
- **Liskov Substitution Principle (LSP):** Instances of `RecurringTask` can seamlessly substitute for base `Task` objects across all manager collection routines without altering runtime correctness.

6.2 System Performance & Memory Efficiency

Operation Module	Time Complexity	Space Complexity	Resource Allocation Note
Task Insertion	O(1) amortized	O(1)	Dynamic allocation via `std::make_unique`
View (Sort by Date)	O(N log N)	O(N)	Temporary vector of raw observation pointers
Keyword Search	O(N × M)	O(1)	Linear scan comparing search token across string buffers
Batch Deletion	O(N)	O(K)	`std::set` lookup table matching K input target IDs

6.3 Limitations & Future Scope

- **Console Formatting:** Future iterations could integrate ANSI escape codes to render colored status badges and table borders directly inside terminal windows.
- **Database Integration:** The file persistence layer could transition from flat delimiter text files (`tasks.txt`) to lightweight SQLite relational database storage.

7. CONCLUSION & ACADEMIC SUMMARY

7.1 Summary of Achievement

The project successfully achieved all specified objectives by constructing a robust, fully functional Command Line To-Do Application in C++. The software effectively bridges theoretical classroom Object-Oriented Programming concepts with practical implementation details. By leveraging encapsulation, class inheritance hierarchies, polymorphic method overriding, strict stream I/O validation, and modern C++ Standard Template Library algorithms, the system delivers a reliable task management solution.

7.2 Learning Outcomes

Through the design and construction of this project, several key software engineering competencies were mastered:

- Designing clean class abstractions that isolate business logic from presentation layers.
- Managing resource ownership safely using modern C++ smart pointers (`std::unique_ptr`).
- Implementing robust input parsing loops that gracefully handle unexpected or malformed user terminal inputs.
- Constructing data serialization pipelines to maintain state persistence across execution sessions.

***** END OF REPORT *****